

CS3780/5780 Practice Prelim

90 minutes, 100 points

Name: _____ NetID: _____

Instructions: You may reference your own notes, but may not use any electronic devices. This exam is individual work; outside communication (e.g., web, email, phones) or outside help must not be used. Show all work on this answer sheet. You have 90 minutes to complete the questions on this exam.

Problem 1: True/False [16 points]

Please identify whether each statement is True or False. If False, give a one-sentence justification. Each statement is worth 2 points.

1. The k-NN classifier is a linear classifier.

True False

2. If you sample points uniformly in a finite high dimensional volume, the most similar point pairs will be almost as far away from each other as the most distant points. This effect is also referred to as the curse of dimensionality.

True False

3. If the data is not linearly separable, the perceptron algorithm still converges to a hyperplane that minimizes the training error.

True False

4. In the soft-margin formulation of Support Vector Machines (SVMs), the value of the regularization parameter C trades off between achieving a lower training error and maximizing the margin, with larger values of C resulting in a stricter classification.

True False

5. The logistic regression objective function (from maximum likelihood estimation) can be solved in a closed form.

True False

6. In ℓ_2 -regularized least squares, the regularization term $\|\theta\|_2^2$, can also be observed by assuming an IID Laplace prior for the model parameters.

True False

7. Any symmetric non-negative function can be used as a positive semi-definite kernel function.

True False

8. Gradient descent with larger learning rate always converges in fewer iterations.

True False

Problem 2: k-Nearest Neighbors [9 points]

1. [9 pts] In class, we saw that as the number of samples approaches ∞ , the error (or risk) of the 1-NN algorithm is at most twice the error of the Bayes optimal classifier for binary classification. In this question, we will examine how the classification error of the KNN algorithm behaves compared to the Bayes error as k increases in the context of binary classification.

Answer the following with justification:

- (a) (3 pts) If we set $k = n$ (number of samples), is the error rate of KNN better than the Bayes error as $n \rightarrow \infty$?

Hint: When $k = n$, what decision rule does KNN use?

- (b) (4 pts) Suppose that at every point x , the label is $+1$ with probability 0.8 and -1 with probability 0.2, so the Bayes optimal classifier always predicts $+1$ and has error 0.2. As $n \rightarrow \infty$, the k nearest neighbors of a test point lie arbitrarily close to it, so their labels behave like independent draws from $P(y | x)$. Compute the asymptotic error of 1-NN and of 3-NN. Which one is closer to the Bayes error, and why?

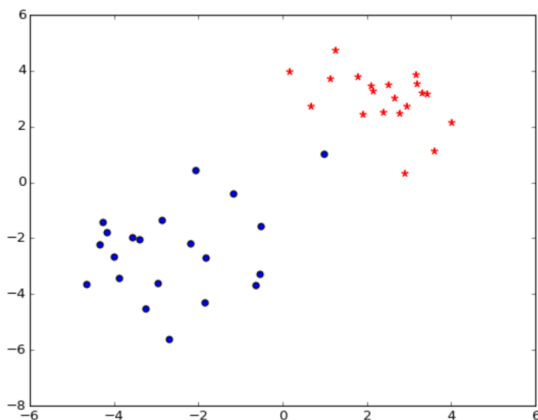
- (c) (2 pts) You have only $n = 40$ training points and want to choose $k \in \{1, 3, 5, 7, 9\}$. (i) Describe a procedure that selects k using only the training data. (ii) Why would picking the k with the lowest *training* error be a bad idea?

Problem 3: Perceptron [10 points]

1. [10 pts] You have a binary classification task with 60,000 binary features (i.e. taking values $\{0, 1\}$). While you don't know which of these features are relevant, you know that at most 50 of those 60,000 features are relevant and that the others tell you nothing about the class label. You also know that for each positive example, no more than 2 of the 50 relevant features take value "1", while for any negative example, at least 6 of the 50 relevant features is set to "1". Finally, each example has at most 99 non-zero features. You only have a Perceptron algorithm for learning homogeneous linear rules, so you add a constant feature that is always "1" to allow for a bias. In this question, you will bound the number of updates that your homogeneous Perceptron will make in the worst case before learning a perfect rule.
 - (a) (2 pts) Calculate an upper bound on $R = \max \|\vec{x}\|_2$ for any sample of data $x \in \{0, 1\}^{60,001}$.
 - (b) (3 pts) Describe a hyperplane $\vec{w}$ for which the classifier $h_{\vec{w}}$ would correctly classify any sample of data.
 - (c) (2 pts) Calculate a lower bound on the geometric margin γ for $\vec{w}$ that holds for any sample of data.
 - (d) (3 pts) Based on the mistake bound R^2/γ^2 , what is the worst-case number of updates the Perceptron will make before learning a perfect rule? For full credit your bound should be as tight as possible; you may need to revise your $\vec{w}$.

Problem 4: Support Vector Machines [12 points]

1. [12 pts] For this question, refer to the plot and dataset below (blue circles and red stars).



- (a) Recall that the soft-margin SVM optimization problem is equivalent to the following:

$$\min_{w,b} w^\top w + C \sum_{i=1}^n \max[1 - y_i(w^\top x_i + b), 0]$$

- (i) (4 pts) Suppose you train an SVM on the dataset above with a very large value for the regularization parameter C (e.g. $C \rightarrow \infty$). Explain what would happen to the learned decision boundary as C increases, and draw as a bold solid line on the plot above a decision boundary most likely to correspond to a very large value of C .
- (ii) (4 pts) Suppose that instead you were to use a small value of C . Explain what would happen to the learned decision boundary as C decreases, and draw as a dashed line on the plot above a decision boundary most likely to correspond to a very small value of C .

- (b) Now let's consider the hard-margin SVM optimization problem which is as follows:

$$\min_{w,b} w^\top w \quad \text{s.t.} \quad \forall i, y_i(w^\top x_i + b) \geq 1$$

- (i) (1 pt) On the plot, draw an additional circle data point that would not affect the decision boundary of a hard margin SVM trained on the dataset.
- (ii) (1 pt) On the plot, draw an additional star data point that would make the hard margin SVM optimization problem infeasible.
- (c) (2 pts) Suppose you hand your dataset to the exciting new startup Bad Machine Learning Solutions Inc., and they give you back a hard margin SVM classifier with parameters w and b . However,

upon inspection, you discover that for every training data point (x_i, y_i) , $y_i(w^\top x_i + b) \geq 2$. After notifying the startup of your discovery, they actually told you that their optimization uses the constraint in the form of $y_i(w^\top x_i + b) \geq 2, \forall i$.

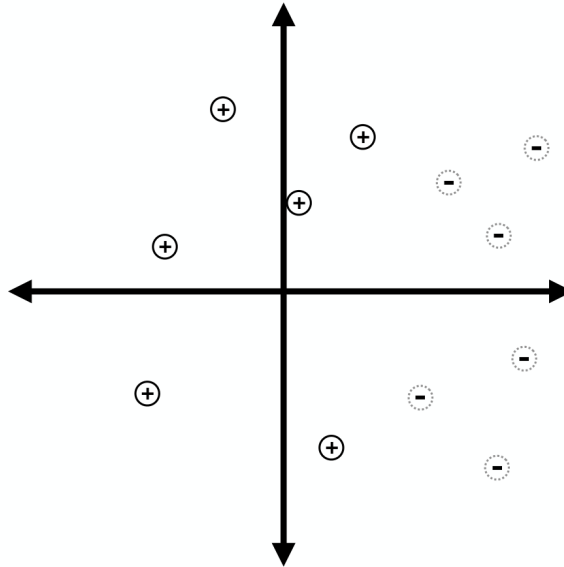
Will the given parameters w and b still give you a classifier that has the maximum margin (briefly explain why or why not)?

Problem 5: Linear Classification [6 points]

1. [6 pts] Tim is training a **linear classifier** to separate two classes of 2D data points of type (+) and (-). Training data is shown in the figure below.

Tim first trains a standard **logistic regression classifier** on the provided data. However, the training procedure fails to converge: the loss keeps decreasing, and the learned weights grow extremely large.

- (a) (4 pts) Explain **why** logistic regression runs into numerical instability on this data.
(b) (2 pts) Suggest one **modification** to the logistic regression algorithm to address this issue.



Problem 6: Linear Regression [15 points]

1. You work as a researcher in hospital A. Your hospital collects a set of patient data to build a model to predict the cost of a treatment for any particular patient. The dataset consists of $x_i \in \mathbb{R}^d$, the feature for patient i , and $y_i \in \mathbb{R}$, the true cost associated with patient i .

You plan to fit a linear regression model $y_i = w^\top x_i + \epsilon_i$ that will enable you to predict a label for future, unseen patients using their feature vectors. Here, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is zero-mean noise. Recall that the 1-dimensional Gaussian distribution is defined as

$$P(z; \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(z-\mu)^2}{2\sigma^2}},$$

where μ and σ^2 are mean and variance, respectively. Assume independent Laplace priors on the model weights, that is,

$$P(w) = \prod_{j=1}^d P(w_j), \quad P(w_j) = \frac{1}{b} e^{-\frac{|w_j|}{b}},$$

where w_j denotes the j th dimension of the weight vector w .

- (a) **[6 pts]** Write out the MAP probability that we wish to maximize. At this step, please do not expand out the individual probabilities, but simplify your answer. Hint: what is the probability MAP is maximizing? Are there terms you can ignore?

- (b) **[6 pts]** Replace each of the probability density functions into your solution above. What objective (the negative log of your expression from (a), dropping terms that do not depend on w) do we minimize to obtain w ?

- (c) **[3 pts]** Rewrite your objective as the following loss and regularizer

$$\frac{1}{n} \sum_{i=1}^n \ell(x_i, y_i, w) + \lambda r(w).$$

What is $\ell()$, what is $r()$, and what is λ ?

Problem 7: Optimization [11 points]

1. [11 pts] Jon has a small dataset of x-y pairs where the value of “y” is sampled based on an unknown linear function $y = x^\top w^* + \epsilon$ (where w^* is unknown and ϵ is an unknown value sampled from a standard normal distribution $\mathcal{N}(\mu = 0, \sigma = 1)$). He wants to train a model to estimate what the value of “y” might be given a particular x , and decides to use linear regression, predicting $\hat{y}$ and optimizing by minimizing the square loss function $L(\hat{y}, y) = (\hat{y} - y)^2$.

Jon decides to use stochastic gradient descent for this, but Jon is working in a weird language that accepts pseudocode instead of Python, so he has to write up the pseudocode himself. He writes the pseudocode as below.

Algorithm 1: Jon’s SGD

Data: $D = \{(x_1, y_1) \dots (x_n, y_n)\}$ where x_i is a training sample and $y_i \in \mathbb{R}$ is its real-valued target

Result: optimized w and b for a linear regressor that can predict y values for training samples

- 1 Initialize weight vector w to 0 vector, bias b to 0, step size η to 1;
- 2 **for** iteration t out of 30 iterations **do**
- 3 Uniformly and randomly draw from the dataset D a batch B of four (sample, label) pairs (x_i, y_i) where i is the index of the pair in the original dataset, s.t. $B = \{(x_i, y_i) : 1 \leq i \leq n\}$;
- 4 Compute predictions $\forall x_i \in B, \hat{y}_i = w^\top x_i + b$;
- 5 Compute loss $\forall y_i \in B, \ell_i = L(\hat{y}_i, y_i)$;
- 6 Update parameters with gradients for each of the samples in the batch:

$$w \leftarrow w + \frac{1}{|B|} \sum_{i \in B} (\eta \cdot \nabla \ell_i(w))$$

$$b \leftarrow b + \frac{1}{|B|} \sum_{i \in B} \left(\eta \cdot \frac{\partial \ell_i}{\partial b} \right)$$

7 **end**

- (a) (3 pts) What’s the batch size for this implementation of SGD? Please write your answer as an integer in the box provided.

- (b) (3 pts) Jon attempts to actually use this algorithm but finds out that his regressor is getting worse and worse over time. He finds that there’s a bug in the algorithm. Which line is the bug in, and what is the change that needs to be made?

- (c) (5 pts) Jon fixes the initial bug in his algorithm and excitedly tries to train on a big dataset D of 1 billion new training samples and labels following the same patterns as before running the same 30 iterations with $\eta = 1$. When he plots the loss curve, he sees a small initial drop, after which the loss jumps up and down around a high value instead of decreasing. Circle all of the reasons Jon’s training might be getting stuck.

- (a) He set his learning rate too high, causing the model to jump past the optimal weight/bias terms over and over again. He can fix this by starting at a lower learning rate, say 0.001, and/or reducing the learning rate over time.
- (b) He's selecting the same samples every time, and overfitting to those samples. His algorithm should be drawing different samples every time.
- (c) His batch size is too small compared to the training data, and he's not making enough passes over the training data for the minibatch gradient to be a useful approximation of the full gradient.
- (d) He's trying to use a linear model for data that is very nonlinear. He should use a different model that better fits the data

Problem 8: Kernel Methods [11 points]

1. [11 pts] Assume you are given a dataset $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, where $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$. Also, assume that the data is linearly separable. If we were to train a classifier on the data, we could run the following algorithm we studied in class:

Algorithm 1: Perceptron Learning Algorithm

Result: We obtain a hyperplane that separates the data.

```
1 Initialize  $w = 0$ 
2 while True do
3      $m = 0$ 
4     for each  $(x_i, y_i)$  in  $D$  do
5         if  $y_i (w^T x_i) \leq 0$  then
6              $w \leftarrow w + y_i x_i$ 
7              $m \leftarrow m + 1$ 
8         end
9     end
10    if  $m = 0$  then
11        break
12    end
13 end
```

Assume now that the data is not linearly separable. We aim to transform the feature representation of our data to capture non-linear interactions among the original features and then train a linear classifier in this new feature space. Formally, for a data vector $x \in \mathbb{R}^d$, we apply the transformation $x \rightarrow \phi(x)$, where $\phi(x) \in \mathbb{R}^D$. Unfortunately, usually $D \gg d$ because we add dimensions that capture non-linear interactions among the original features. To tackle this challenge, we will use the powerful trick we learned in class: kernels. This approach will enable us to train a classifier for data that is not linearly separable, without ever computing high-dimensional vectors like w or $\phi(x)$. Use the following subquestions to structure your proof that the perceptron algorithm is indeed kernelizable.

- (a) First, prove that the perceptron algorithm returns a vector w that lives in the span of the data, i.e. that there exists a parameter $\alpha \in \mathbb{R}^n$ that expresses w as a linear combination of all input vectors. (We will call α the parameter defining the dual representation of the perceptron algorithm.)
Hint: you could use induction.

- (b) Now, if you remember, to apply the kernel trick, all we have left to do is to check whether this modified formulation of the perceptron learning algorithm “works only by computing inner products between examples”. How would you fill out Algorithm 2 below to obtain the modified perceptron algorithm? Only use $(x_1, \dots, x_n)$ and $(\alpha_1, \dots, \alpha_n)$. Make sure to write the first update step by computing inner products between examples.

Algorithm 2: Modified Perceptron Learning Algorithm

Result: We obtain a hyperplane that separates the data.

```

1 Initialize _____ ;
2 while True do
3     m = 0
4     for each (x_i, y_i) in D do
5         if _____ ≤ 0 then
6             _____
7             m ← m + 1
8         end
9     end
10    if m = 0 then
11        break
12    end
13 end

```

- (c) Now, let us assume that we have a feature map $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that $\langle \phi(x_i), \phi(x_j) \rangle = K(x_i, x_j)$ and K is very easy to compute. How would you fill out the algorithm below in Algorithm 3 to obtain the kernelized modified perceptron algorithm?

Algorithm 3: Kernelized Perceptron Learning Algorithm

Result: We obtain a hyperplane that separates the data.

```

1 Initialize _____ ;
2 while True do
3     m = 0
4     for each (x_i, y_i) in D do
5         if _____ ≤ 0 then
6             _____
7             m ← m + 1
8         end
9     end
10    if m = 0 then
11        break
12    end
13 end

```

Problem 9: Model Selection [10 points]

1. [10 pts] You join an email security company fighting a surge in spam emails. You build a binary classifier that labels each incoming message as “spam/not”. You’ve curated ground-truth labels for 30,000 randomly sampled emails and engineered an input representation with 25,000 features. The product team will ship your model only if its prediction error is below 2%. You design an experiment and a statistical test to rigorously decide whether your learned classifier achieves this performance. After training and evaluation, your measured test error is 1.8%.

Fill in the blanks and be as specific with your answers as possible:

Randomly select 5000 of the 30000 examples and put them into the _____ sample, which you will not touch until the very end. The remaining examples you further split into a _____ sample of 20000 examples and a _____ sample of 5000 examples. You train classifiers with different hyperparameters and perform model selection for selecting the classifier that has best performance on the _____ sample. You then use that learned classifier to classify the _____ sample. You record the number of _____, which you know follows a _____ distribution with parameter p equal to the _____ for i.i.d. data. You then test the null _____ that p is _____.